

C 语言错题本 · 第一批

整理日期：2026-06-14
内容来源：期末复习错题截图

目录

- 1. 运算符优先级
- 2. 整数除法
- 3. 进制表示
- 4. scanf 输入格式
- 5. 逗号运算符
- 6. 函数定义规则
- 7. 转义字符
- 8. if-else 语法陷阱
- 9. if 条件中的赋值

1. 运算符优先级

还记得我上一条消息给你总结的优先级口诀吗？
算术 > 关系 > 逻辑 > 赋值

我们把这四个选项对号入座：

- **B. % (取余)**：属于**算术运算符**，优先级最高！（和乘法 *****、除法 **/** 是同一级别的）
- **C. <= (小于等于)**：属于**关系运算符**，排第二。
- **D. && (逻辑与)**：属于**逻辑运算符**，排第三。
- **A. = (赋值)**：属于**赋值运算符**，优先级最低（垫底）。

所以，这组里老大是 **%**，记得把答案改成 **B** 哦！

口诀

算术 > 关系 > 逻辑 > 赋值

选项分析示例

运算符	类型	优先级
%	算术运算符	最高

运算符	类型	优先级
<=	关系运算符	第二
&&	逻辑运算符	第三
=	赋值运算符	最低（垫底）

结论：`%` 与 `*`、`/` 同级，在这组选项中优先级最高 → 选 B。

2. 整数除法

你是不是不小心把 `5/2` 算成 `2.5` 啦？或者心算加减法的时候手滑了？
一定要牢记 C 语言的这个大坑：整数除以整数，结果还是整数（直接抹掉小数部分，不四舍五入）。
`5 / 2` 的结果就是 `2`，绝对不能带入 `2.5` 去算。

⚠ 避坑指南

C 语言中：`整数 ÷ 整数 = 整数`，小数部分直接截断，不四舍五入。

```
5 / 2 // 结果是 2，不是 2.5！
```

常见错误

- 心算时把 `5/2` 当成 `2.5` 继续计算
- 加减法手滑

牢记：只要两边都是整数，结果一定是整数。

3. 进制表示

3. C 语言中的进制表示法（重点排坑）

在 C 语言（以及 Python 等大多数语言）的代码中，为了区分你写的一个数字到底是什么进制，编译器规定了特定的前缀：

进制	C 语言前缀	代码示例	对应的十进制值	备注
十进制	无	<code>int a = 10;</code>	10	默认情况
八进制	数字 0 开头	<code>int b = 010;</code>	8	极易踩坑！看到 0 开头一定要警惕
十六进制	0x 或 0X 开头	<code>int c = 0x1A;</code>	26	内存地址、颜色代码常用
二进制	0b 或 0B 开头	<code>int d = 0b101;</code>	5	C23标准正式引入，GCC扩展常用

C 语言中各进制的写法

进制	前缀	代码示例	十进制值	备注
十进制	无	<code>int a = 10;</code>	10	默认情况
八进制	数字 0 开头	<code>int b = 010;</code>	8	⚠️ 极易踩坑！看到前导 0 要警惕
十六进制	0x 或 0X	<code>int c = 0x1A;</code>	26	常用于内存地址、颜色值
二进制	0b 或 0B	<code>int d = 0b101;</code>	5	C23 标准正式支持，GCC 扩展常用

⚠️ 避坑指南

- 010 不是十进制的 10，而是八进制的 8
- 二进制字面量在较新编译器中才支持

4. scanf 输入格式

1. %d 家族有“物种隔离”

`%d` 里的 `d` 代表的是十进制整数 (decimal integer)。它就像一个只吃素的胃，只认识数字 `0-9` 和正负号 `+ -`，它根本不认识小数点 `.`！

如果你想读小数，必须换成 `%f` (对应 `float`) 或者 `%lf` (对应 `double`)。

2. 如果硬输入 9.9 会发生什么可怕的事？

假设你的代码是 `scanf("%2d", &i);`，然后你在键盘上敲了 `9.9`。`scanf` 的反应会非常戏剧性：

1. 它先看到第一个字符 `9`，符合整数要求，收下。
2. 接着它看到了小数点 `.`。这时候它会大喊一声：“这不是整数！”然后立刻罢工，停止读取（哪怕它还没读满 2 个字符）。
3. 结果：你的变量 `i` 只被赋值了 `9`。
4. 灾难后果：那个小数点和后面的 `9`（也就是 `.9`）并没有消失，它们会像垃圾一样死死卡在输入缓冲区里。如果你后面还有其他的 `scanf`，它们一上来就会撞到这个小数点，导致后面的输入全部崩溃乱套！

总结一下：

`%2d` 只能用来限制整数的长度。一旦输入里混进了小数点、字母或者其他奇怪的符号，`scanf` 就会立刻“翻车”停止读取。

4.1 %d 的“物种隔离”

- `%d` 中的 `d` = **decimal integer**（十进制整数）
- 只认 `0-9` 和 `+`、`-`，不认小数点 `.`
- 读小数要用 `%f` (float) 或 `%lf` (double)

4.2 硬输入 9.9 时会发生什么？

代码：`scanf("%2d", &i);`；键盘输入 `9.9`

步骤	发生了什么
1	读到 <code>9</code> ，合法，收下
2	遇到 <code>.</code> ，立即停止（即使 <code>%2d</code> 允许读 2 位）
3	<code>i = 9</code>
4	灾难： <code>.9</code> 留在输入缓冲区，后续 <code>scanf</code> 会读到它，程序乱套

总结：`%2d` 限制最多读 2 位整数；遇到小数点、字母等“奇怪符号”会立刻“罢工”。

4.3 %2d 限制的是字符数，不是数值大小

 `scanf %2d` 宽度

- 负号 - 也占 1 个字符位

输入	字符数	%2d 读取结果	剩余在缓冲区
-5	2 (- 和 5)	-5	无
-15	3 (-、1、5)	-1	5 留在缓冲区

结论：%2d 能完整读入的范围是 -9 到 99。

5. 逗号运算符

- 例子：`a = 1, a++, a = a * 2`
 - 先执行 `a = 1`
 - 再执行 `a++` (此时 a 变成 2)
 - 最后执行 `a = a * 2` (此时 a 变成 4)

铁律 2：最终结果“最后一个说了算”

这是逗号表达式最核心的特性！虽然前面的一大串表达式都执行了，但整个逗号表达式的最终结果，永远等于最后一个（最右边）表达式的值。

- 例子：`int result = (3+5, 10*2, 8-5);`
 - 电脑会偷偷算 `3+5`，算 `10*2`，但算完就扔了。
 - 最后算 `8-5 = 3`。
 - 所以，整个括号里的结果就是 3，`result` 会被赋值为 3。

铁律 3：优先级“全场垫底”

这是你在第 16 题踩坑的根本原因！

在 C 语言所有的运算符（加减乘除、逻辑运算、赋值运算等）里面，逗号运算符 `,` 的优先级是最低的，倒数第一！

我们再来对比一下加了括号和没加括号的致命区别：

场景 A（没加括号，第 16 题的陷阱）：

```
z = x++, y++, ++y;
```

因为 `=` 的优先级比 `,` 高，所以电脑会优先把 `z = x++` 绑定在一起。

实际执行的是：(1) 先把 `x++` 赋值给 `z`，(2) 然后算 `y++`，(3) 最后算 `++y`。

(这里逗号的作用仅仅是把三个动作隔开，并没有把最后的值赋给 `z`)。

铁律 1：从左到右依次执行

```
a = 1, a++, a = a * 2;  
// 执行顺序: a=1 → a++(a变2) → a=a*2(a变4)
```

铁律 2：最终结果是"最后一个说了算"

```
int result = (3+5, 10*2, 8-5);  
// 3+5 和 10*2 算了但丢弃，最终 result = 8-5 = 3
```

铁律 3：优先级全场最低

逗号运算符优先级比赋值、逻辑、算术都低。

场景 A：没加括号的陷阱

```
z = x++, y++, ++y;
```

因为 `=` 优先级高于 `,`，实际等价于：

```
(z = x++), (y++), (++y);  
// z 拿到的是 x++ 的值，不是 ++y 的值！
```

场景 B：加了括号

```
z = (x++, y++, ++y);  
// z 拿到的是 ++y 的值
```

6. 函数定义规则

- **A 错**：函数绝对不能在 `main()` 里面定义！C 语言不允许“函数嵌套定义”，所有函数（包括 `main`）都是平级的，大家都在外面定义。
- **B 错**：`main()` 函数可以放在代码的任何位置，最前面、中间、最后面都行。
- **C 错**：电脑不管你第一个写的是什么函数，它运行程序时，就像带了 GPS 一样，只会满世界找 `main` 这个名字，找到它才开始执行。所以，**D 是 C 语言的铁律**！

A 错：函数不能在 `main()` 里面定义

C 语言**不允许函数嵌套定义**。所有函数（包括 `main`）都是平级的，都在外面定义。

B 错：`main()` 可以放在代码任何位置

最前面、中间、最后面都行，位置不影响功能。

C 错：程序入口永远是 `main`

电脑不管你第一个写的是什么函数，运行时只会找 `main` 这个名字，找到才开始执行。

D 正确：C 语言的铁律

程序从 `main` 函数开始执行。

7. 转义字符

第一类：排版控制类（“打字机”指令）

这类字符在屏幕上是不见的，它们的作用是控制光标的移动。

转义字符	名称	实际效果（魔法作用）	考试常考坑点
<code>\n</code>	换行 (Newline)	光标跳到下一行的开头。	最常用，没啥坑。
<code>\t</code>	制表符 (Tab)	光标跳到下一个 Tab 停靠点（通常是对齐用的，相当于按了键盘上的 Tab 键）。	经常用来做表格对齐。
<code>\b</code>	退格 (Backspace)	光标往回退一格。	大坑！退格本身不删字符，但如果退格后接着输出新字符，就会覆盖掉原来的字符。（就是你刚做错的第 24 题）
<code>\r</code>	回车 (Return)	光标直接回到当前行的最开头。	大坑！如果写 <code>printf("123\r45");</code> ，光标打完 123 后回到行首打 45，结果会变成 453！

第二类：符号输出类（防冲突专属）

在 C 语言里，单引号 `'`、双引号 `"` 和反斜杠 `\` 都有特殊的语法作用（比如双引号用来包裹字符串）。如果你想在屏幕上单纯地打印出这些符号，就必须用魔法棒 `\` 把它们“打回原形”。

转义字符	实际打印出的符号	为什么要转义？
<code>\\</code>	<code>\</code> (反斜杠)	因为 <code>\</code> 是魔法棒，你想打印一根普通的魔法棒，就得用另一根魔法棒点它一下。
<code>\'</code>	<code>'</code> (单引号)	防止和字符的边界（如 <code>'A'</code> ）混淆。
<code>\"</code>	<code>"</code> (双引号)	防止和字符串的边界（如 <code>"Hello"</code> ）混淆。

第一类：排版控制类（"打字机"指令）

屏幕上不可见，控制光标移动。

转义字符	名称	实际效果	考试易错点
<code>\n</code>	换行	光标跳到下一行开头	最常用，无大坑

转义字符	名称	实际效果	考试易错点
<code>\t</code>	制表符	光标跳到下一个 Tab 位，用于对齐	常用来对齐表格
<code>\b</code>	退格	光标后退一格	⚠️ 退格本身不删字符，但后面输出新字符会覆盖原字符
<code>\r</code>	回车	光标回到 当前行最开头	⚠️ <code>printf("123\r45")</code> → 先打 123，光标回到行首，再打 45 覆盖 12，最终显示 453

第二类：符号输出类（防冲突）

用 `\` 这根"魔杖"把特殊符号还原成普通符号。

转义字符	实际打印	为什么要转义
<code>\\</code>	<code>\</code>	<code>\</code> 是魔杖，要打印普通反斜杠需再敲一下
<code>\'</code>	<code>'</code>	防止和字符常量边界混淆（如 <code>'A'</code> ）
<code>\"</code>	<code>"</code>	防止和字符串边界混淆（如 <code>"Hello"</code> ）

第三类：终极 Boss（字符串的"休止符"）

转义字符	名称	核心作用
<code>\0</code>	空字符	ASCII 值为 0，C 字符串的隐藏结束标记

⚠️ 避坑指南

```
"abc"           // 占 4 字节: 'a' 'b' 'c' '\0'
strlen("abc")   // 返回 3（只数可见字符）
sizeof("abc")    // 返回 4（含 '\0' 的总内存）
```

第四类：数字转义（考试高频坑）

第三类：终极 Boss（字符串的“休止符”）

转义字符	名称	核心作用
<code>\0</code>	空字符 (Null)	它的 ASCII 码值就是数字 0。它是所有 C 语言字符串的隐藏结尾！

💡 避坑指南：

当你写 `"abc"` 的时候，它在内存里其实占了 4 个字节，分别是 `'a'`，`'b'`，`'c'`，和系统偷偷加上的 `\0`。

`strlen("abc")` 算的是看得见的字符，结果是 3。

`sizeof("abc")` 算的是内存大小（包含 `\0`），结果是 4！

第四类：数字转义类（考试最爱挖的“乱码坑”）

你可以直接用数字来打印出对应的 ASCII 字符。这里有两种写法：

1. 八进制转义： `\ddd`
- 规则： `\` 后面跟 1 到 3 位八进制数字 (0~7) 。
 - 例子： `\101` 。八进制的 101 换算成十进制是 65，也就是大写字母 `'A'` 。
 - 🚫 致命陷阱：八进制里绝对不可能出现数字 8 和 9！如果考试出 `\08` 或者 `\19`，这绝对是非法的转义字符！
2. 十六进制转义： `\xhh`
- 规则： `\x` 后面跟 1 到 2 位十六进制数字 (0~9, A~F) 。
 - 例子： `\x41` 。十六进制的 41 换算成十进制也是 65，也就是大写字母 `'A'` 。
 - 🚫 致命陷阱：必须以小写的 `x` 开头，不能省略！

八进制转义 `\ddd`

- 规则： `\` 后跟 1~3 位八进制数 (0~7)
- 示例： `\101` → 八进制 101 = 十进制 65 = 字符 `'A'`
- ⚠️ 致命陷阱：八进制不能含 8 和 9，`\08`、`\19` 是非法转义字符

十六进制转义 `\xhh`

- 规则： `\x` 后跟 1~2 位十六进制数 (0~9, A~F)
- 示例： `\x41` → 十六进制 41 = 十进制 65 = 字符 `'A'`
- ⚠️ 致命陷阱：必须以小写 `x` 开头，不能省略也不能大写

8. if-else 语法陷阱

🔥 第 27 题：硬核的“棒打鸳鸯”陷阱

题目代码：

c

📄 复制

```
int a=0,b=0,c=0,d=0;
if(a=1) b=1;c=2;
else d=3;
printf("%d,%d,%d,%d\n",a,b,c,d);
```

你选了 D (1,1,2,0)，你的思路肯定是这样的：`a=1` 是赋值，结果为真，所以执行 `b=1` 和 `c=2`，跳过 `else`。

但是，这道题的正确答案是 C（编译有错）！

破案关键：请回忆我们之前讲过的“`if` 不加大括号，只能管一句”的铁律！

我们把这段代码按照电脑的理解，加上大括号还原一下：

c

📄 复制

```
if (a=1) {
    b=1; // if 只能管到第一个分号结束
}
c=2;    // 这是一句独立的代码，跟 if 没有任何关系！
else {  // 报错！！！
    d=3;
}
```

发现问题了吗？在 C 语言里，`else` 必须紧紧贴着 `if` 的屁股后面，它们俩是一对不可分割的组合。

但是在这里，中间硬生生插进来一个“第三者”`c=2; !`

电脑读到 `else` 的时候会一脸懵逼：“我前面明明是 `c=2;`，没有 `if` 呀，这个 `else` 是从哪里冒出来的？”于是，编译器直接罢工，报出经典错误：`'else' without a previous 'if'`（`else` 找不到匹配的 `if`）。

💡 避坑总结：

只要看到 `if` 和 `else` 之间夹杂了两个或以上以分号结尾的语句，且没有用 `{ }` 包起来，不用犹豫，绝对是编译错误！

题目 27：硬核“拆散情侣”陷阱

```
int a=0, b=0, c=0, d=0;
if(a=1) b=1; c=2;
else d=3;
printf("%d,%d,%d,%d\n", a, b, c, d);
```

你以为是输出

`a=1` 为真 → `b=1` 和 `c=2` 都执行 → 输出 `1,1,2,0`

正确答案

编译错误!

破案关键

`if` 没有花括号 `{}` 时, 只能控制紧跟其后的那一个语句 (到第一个分号为止)。

编译器实际看到的结构:

```
if (a=1) {  
    b=1;    // if 只控制到这里  
}  
c=2;        // 独立语句, 和 if 无关  
else {      // 错误!!!  
    d=3;  
}
```

`else` 必须紧跟在 `if` 控制的语句之后。中间夹了 `c=2;`, `else` 找不到匹配的 `if`, 报错:

'else' without a previous 'if' (else 找不到匹配的 if)

⚠ 避坑总结

只要看到 `if` 和 `else` 之间有两条以上语句 (两个分号), 且没用 `{}` 包裹 → 一定是编译错误。

9. if 条件中的赋值

1. 千万别选“语法错误”!
2. 直接算等号右边的值!
3. 右边是 0 就是假 (走 else), 右边不是 0 就是真 (走 if) !

三句口诀 (应对 `if(a=0)` 类题目)

1. 千万别选“语法错误” — `if(a=0)` 语法完全合法
2. 直接算等号右边的值 — `if(a=5)` 整个表达式的值就是 5
3. 右边是 0 就是假 (走 else), 不是 0 就是真 (走 if)

示例

```
if (a = 0)    // a 被赋值为 0, 条件为假 → 走 else  
if (a = 5)    // a 被赋值为 5, 条件为真 → 走 if
```

⚠ 避坑指南

- `=` 是赋值, `==` 才是比较, 但 `if(a=5)` 不是语法错误
- 条件判断看的是赋值后右边的值, 不是比较结果

速记卡片

知识点	一句话记住
运算符优先级	算术 > 关系 > 逻辑 > 赋值
整数除法	整数 ÷ 整数 = 整数, 截断不四舍五入
八进制	<code>010</code> = 8, 不是 10
<code>scanf %d</code>	不认小数点, 剩余字符留缓冲区
<code>%2d</code>	限字符数, 负号占一位, 范围 -9~99
逗号运算符	从左执行, 结果为最后一个, 优先级最低
函数定义	不能嵌套, <code>main</code> 位置随意, 入口永远是 <code>main</code>
<code>\r</code>	回车到行首, " <code>123\r45</code> " → 453
<code>\0</code>	字符串结束符, <code>sizeof</code> 比 <code>strlen</code> 多 1
<code>if</code> 无花括号	只控制一条语句, <code>else</code> 中间不能夹语句
<code>if(a=0)</code>	不是语法错误, 看右边值是否为 0

附录：原始截图索引

所有图片保存在 `images/第一批/` 目录下：

文件名	对应知识点
<code>01_运算符优先级.png</code>	算术 > 关系 > 逻辑 > 赋值
<code>02_整数除法.png</code>	$5/2=2$, 整数除法截断
<code>03_进制表示.png</code>	十/八/十六/二进制写法
<code>04_scanf%d输入.png</code>	<code>%d</code> 物种隔离、输入 9.9 的灾难
<code>05_scanf%2d宽度.png</code>	<code>%2d</code> 限字符数, 范围 -9~99
<code>06_逗号运算符.png</code>	逗号运算符三条铁律
<code>07_函数定义规则.png</code>	不能嵌套、 <code>main</code> 位置、入口
<code>08_转义字符_排版与符号.png</code>	<code>\n \t \b \r</code> 及符号转义
<code>09_转义字符_空字符与数字转义.png</code>	<code>\0</code> 、 <code>\ddd</code> 、 <code>\xhh</code>
<code>10_if-else无花括号陷阱.png</code>	<code>else without previous if</code>

文件名	对应知识点
11_if条件赋值口诀.png	if(a=0) 三句口诀
12_if-else陷阱_重复.png	(与 10 重复)
13_if赋值口诀_重复.png	(与 11 重复)

下一批照片发来后，会继续追加到本错题本，图片存入 *images/ 第二批/*。